SOFTWARE ARCHITECTURE — CHAPTER 7

# Composability and Connectors

How components are composed into systems and how they communicate

Based on Pautasso — Visual Lecture Notes

RECAP — LECTURE 6

# Reusability & Interfaces

Chapter 6: how we shape components so they can be reused and evolved

Examples: `example1_interface_design_and_contracts.py`, `example2_reusability_and_versioning.py`

→ Today: how reusable parts are **composed** with **connectors** (Chapter 7).

# Recap: interfaces & contracts

**Good interfaces** are **minimal** (only what clients need), **stable** (avoid breaking changes), and **clear** (consistent naming and semantics).

- **Abstraction / information hiding** — expose behavior, hide implementation details
- **Contracts** — preconditions (what must hold before a call), postconditions (what holds after)
- Multiple implementations behind one interface (e.g. cache backends) = real reusability

# Recap: API design

- **Consistency** — patterns for naming, errors, and resources across endpoints
- **Discoverability** — docs, predictable URLs or RPC names
- **Backward compatibility** — don’t surprise existing clients when you extend

Lecture 6 assignment tied this to exposing a library as an **HTTP/REST API** (layers, request/response shapes).

# Recap: library vs framework

| | Library | Framework |
|---|---|---|
| Control flow | **You** call it | It **calls you** (inversion) |
| Reuse | Functions / classes | Handlers, plug-ins, hooks |

Same component can be packaged as a library or hosted inside a framework — the interface story changes.

# Recap: evolution & versioning

- Extend with **optional parameters** (defaults) instead of breaking signatures
- **Deprecate** clearly (warnings, docs), remove in a later major release
- **Semantic versioning**: MAJOR = breaking, MINOR = compatible features, PATCH = fixes

Plan a **migration path** when interfaces must change.

# Learning objectives

- Compose components into larger systems
- Design and choose **connectors** (how components talk)
- Understand composition patterns and architectural connectors
- Use **message passing** (sync vs async)
- Design **event-driven** compositions
- Compare **orchestration** vs **choreography**

# What is composition?

**Composition** = building a system by assembling smaller **components** rather than one monolithic block.

- Each component has a clear responsibility
- Components are connected by **connectors**
- Architecture ≈ **components + connectors**

You reuse and replace parts; the whole is defined by how parts interact.

# Connectors

A **connector** is the mechanism that allows communication between components.

- Defines *protocol*, *direction*, and often *sync vs async*
- Same component can use different connectors to different neighbors
- Choosing the wrong connector affects coupling, latency, and failure modes

# Connector types (overview)

| Type | Typical use | Often |
|---|---|---|
| Direct call | Same process, in-memory | Sync |
| RPC / REST | Services over network | Sync (request/response) |
| Message queue | Decouple producer/consumer | Async |
| Event bus | Many subscribers, pub/sub | Async |
| Shared DB | Implicit coupling via data | Sync read/write |

# Message passing

### Synchronous

Caller sends a message and **waits** for a reply.

Simple mental model; tight temporal coupling; blocking under load.

### Asynchronous

Caller sends and **continues**; consumer processes later.

Better decoupling and scale; harder reasoning and debugging.

# When to use sync vs async?

- **Sync** when you need an immediate result to proceed (e.g. validate payment before shipping)
- **Async** when work can be deferred (e.g. send email, generate report)
- Async often uses queues or events to absorb spikes and isolate failures

# Composition patterns

- **Pipeline** — output of A is input of B (sequential stages)
- **Layered** — upper layer calls lower layer only
- **Blackboard / shared state** — components read/write shared data (use with care)
- **Microservices / SOA** — services composed via APIs and messaging

# Service composition

Multiple services cooperate to fulfill one business capability.

- Each service exposes a contract (API or events)
- Composition can be **orchestrated** or **choreographed**
- Cross-cutting concerns: auth, tracing, idempotency, timeouts

# Orchestration

One component (the **orchestrator**) **controls the flow**: it calls other services in order and handles branching.

- **Pros:** Flow is visible in one place; easier to change sequence
- **Cons:** Orchestrator can become a bottleneck; couples flow to one component

# Choreography

No central coordinator: each service reacts to **events** (or messages) and may publish new ones.

- **Pros:** Loose coupling; easy to add new subscribers
- **Cons:** Global flow is implicit; harder to trace and test end-to-end

# Orchestration vs choreography

| | Orchestration | Choreography |
|---|---|---|
| Control | Central | Distributed |
| Visibility | High (one place) | Lower (many handlers) |
| Scaling | Orchestrator matters | Workers scale independently |
| Good for | Clear workflows | Many reactive consumers |

# Event-driven architecture

- **Event** — something happened (past tense: `OrderPlaced`)
- **Event bus / broker** — delivers events to subscribers
- Enables choreography, async processing, and integration without tight API coupling
- Watch for: ordering, duplicates, eventual consistency, schema evolution

# Architectural connectors

At the architecture level, connectors are not just “function calls”:

- They carry **quality attributes**: reliability, latency, security
- They appear in **deployment** and **runtime** views
- Document them in diagrams (REST, AMQP, gRPC, WebSocket, ...)

# Course examples (Python)

- `example1_composition_and_connectors.py` — order pipeline, connector types, sync vs async
- `example2_orchestration_vs_choreography.py` — orchestrator vs event bus choreography

Run them after the lecture to reinforce concepts.

ASSIGNMENT

# Composable Document Processing Pipeline

Apply Chapter 7: components, connectors, orchestration vs choreography

Full spec: `ASSIGNMENT.md` — this section orients you; it does **not** solve the assignment.

# Assignment: scenario

Design the **architecture** of a pipeline that:

- **Ingests** documents (e.g. PDF, images) with optional options
- **Runs** conceptual steps: validate → extract (text/OCR) → classify → store → notify
- **Serves** clients needing **sync** (“process and return now”) and **async** (“job id, notify when done”)

You document **design choices**, not production code.

# What you produce (big picture)

| Area | You decide & justify |
|---|---|
| Components | ≥5, responsibilities, I/O, sync/async view |
| Connectors | Types, protocols, where sync vs async |
| Styles | One **orchestrated** and one **choreographed** design + comparison |
| API | Sync + async client entry points |
| Diagrams | Component-and-connector + one sequence diagram |

# Part 1 — Components & connectors

**Task 1.1** (written)

- **≥5 components** — each: name, single responsibility, inputs/outputs, caller's sync vs async
- **Connectors** between pairs: name, type (call, REST, queue, bus, ...), sync/async, format
- **Justify** sync vs async (e.g. fast feedback vs deferred heavy work)

Deliverable: `part1_components_and_connectors.md` — **30 pts**

# Part 1 — Diagram

**Task 1.2**

- **draw.io** component-and-connector diagram: boxes + **labeled** arrows (connector type, sync/async)
- Include a **legend**

Deliverables: `.drawio` and `.png` — **20 pts**

# Part 2 — Orchestration

**Task 2.1**

- Which component **orchestrates**; **order** of calls to others
- **Errors / retries** (conceptually)
- **One** advantage and **one** disadvantage for *your* pipeline

Deliverable: `part2_orchestration.md` — **15 pts**

# Part 2 — Choreography

**Task 2.2**

- **Event** names for the lifecycle (you define the set)
- Per component: what it **subscribes** to and what it **publishes** and when
- **No** central orchestrator — flow emerges from events
- **One** pro and **one** con vs your orchestrated design

Deliverable: `part2_choreography.md` — **15 pts**

# Part 2 — Compare & recommend

**Task 2.3**

- Short **comparison table**: e.g. changing order, adding steps, debugging/tracing, latency & scale (conceptual)
- **Recommend** orchestration, choreography, or **hybrid** — brief justification

Deliverable: `part2_comparison.md` — **10 pts**

# Part 3 — API (sync & async)

**Task 3.1**

- **Endpoints** (REST or RPC-style): e.g. sync “run now” vs async “create job” + status (optional webhook)
- Map **sync path** and **async path** to *your* components/connectors (queues/events as you designed)

Deliverable: `part3_api_design.md` — **20 pts**

# Part 3 — Sequence diagram

**Task 3.2**

- **One** end-to-end flow: **either** full sync **or** async (enqueue → worker → … → notify)
- **≥5 participants**, labeled messages

Deliverables: `part3_sequence_diagram.drawio` + `.png` — **10 pts**

# Submission & grading

- **Pull Request** to `lecture-7/submissions/YOUR_NAME/` — see `../lecture-3/SUBMISSION_GUIDE.md`
- **Total: 120 points** — full rubric in `ASSIGNMENT.md`
- Every diagram: **both** source ( `.drawio` ) and `.png`

# Before you start

Hints — not answers

1. Re-read the **two Python examples** in this lecture folder
2. List **pipeline steps**, then **assign** steps to components (your split)
3. Pick a **connector** per link: sync when you must wait; async when you can defer
4. Sketch **orchestrator call chain**, then **events + subscribers** for the other style

# Takeaways

- Systems are **composed** of components linked by **connectors**
- Choose connector type and sync/async to match requirements and trade-offs
- **Orchestration** centralizes flow; **choreography** distributes it via events
- Event-driven design supports scalable, loosely coupled composition

# Questions?

Chapter 7 — Composability and Connectors

Next: Chapter 8 — Compatibility and Coupling